

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.: 52	Name: Divyanjali yadav
Class: TYIT	Batch:2
Date of Assignment: 21-07-2026	Date/Time of Submission:28-07-2026

a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Step1: Start the application.

Step2: Create a web form with a TextBox and Buttons.

Step3: Enter the user's name in the TextBox.

Step4: Create an HTTP Cookie and store the entered value.

Step5: Add the cookie to the Response object.

Step6: Redirect to the second web page.

Step7: Read the cookie using the Request object.

Step8: Display the cookie value (Welcome message).

Step9: Stop.

Code:

[webform1.aspx.cs](#)

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace pract5
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (IsPostBack)
            {
                int viewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
                Label2.Text = "view State = " + viewStateVal.ToString();
            }
            else
            {
                ViewState["count"] = 1;
            }
        }
    }
}
```

```

protected void Button1_Click(object sender, EventArgs e)
{
    Label1.Text = ViewState["count"].ToString();
}

protected void Button2_Click(object sender, EventArgs e)
{
    HttpCookie h = new HttpCookie("name");
    h.Value = TextBox1.Text;
    Response.Cookies.Add(h);
    Response.Redirect("WebForm2.aspx");
}
}
}

```

[webform2.aspx.cs](#)

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace pract5
{
    public partial class WebForm2 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (Request.Cookies["name"] != null)
            {
                Response.Write("Welcome " + Request.Cookies["name"].Value);
            }
        }
    }
}

```

Output:

The screenshot shows the rendered web page. It contains two labels, both displaying the text "Label". Below the first label is a button labeled "View State". Underneath the button is a text input field. At the bottom of the visible area is another button labeled "Cookies".

1

view State = 2

View State

divyanjali

Cookies

Welcome divyanjali

b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

Step1: Start the application.

Step2: Create the Product model with Id, Name, and Price.

Step3: Configure the database connection in `appsettings.json`.

Step4: Create `AppDbContext` and register it in `Program.cs`.

Step5: Create the Product table using Entity Framework Core.

Step6: Create a Product object with required details.

Step7: Add the Product object to the database context.

Step8: Save the changes using `SaveChanges()`.

Step9: Display "Product Inserted Successfully".

Code:

Step 1: Product Model ([Product.cs](#))

```

1  namespace ProductEFCore.Models
2  {
3      3 references
4      public class Product
5      {
6          0 references
7          public int Id { get; set; }
8          1 reference
9          public string Name { get; set; }
10         1 reference
11         public int Price { get; set; }
12     }
13 }

```

Step 2: AppDbContext ([AppDbContext.cs](#))

```

using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;
7 references
public class AppDbContext: DbContext
{
    0 references
    public AppDbContext(DbContextOptions<AppDbContext> options) : base (options)
    {
    }
    1 reference
    public DbSet<Product> Products { get; set; }
}

```

Step 3: appsettings.json

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True"
  }
}

```

Step 4: [Program.cs](#)

```

using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
name: "default",
pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

Step 6: HomeController

```

1 reference
public class HomeController : Controller
{
    private readonly ApplicationDbContext _context;
    0 references
    public HomeController(ApplicationDbContext context)
    {
        _context = context;
    }
    0 references
    public IActionResult Index()
    {
        Product p = new Product();
        p.Name = "Laptop";
        p.Price = 50000;
        _context.Products.Add(p);
        _context.SaveChanges();
        return Content("Product Inserted Successfully");
    }
}

```

Output:

Product Inserted Successfully

```

1 select * from products;
2

```

	Id	Name	Price
1	1	Laptop	50000

C) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on the Employee table using EF Core.

Algorithm:

Step1: Start the application.

Step2: Create the Employee model.

Step3: Configure SQL Server connection and **DbContext**.

Step4: Create the Employee table using EF Core.

Step5: Insert a new employee record (Create).

Step6: Retrieve and display employee records (Read).

Step7: Edit and update an existing employee record (Update).

Step8: Delete the selected employee record (Delete).

Step9: Save changes to the database.

Step10: Display the updated employee list.

Step11: Stop.

Code:

[Program.cs](#)

```
using Microsoft.EntityFrameworkCore;
```

```

using EmployeeCRUD.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();
AppDbContext.cs
using EmployeeCRUD.Models;
using Microsoft.EntityFrameworkCore;

namespace EmployeeCRUD.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
Employee.cs
namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Salary { get; set; }
    }
}

```

Output:

← → ↻  https://localhost:7064

EmployeeCRUD Home Privacy

Welcome

Learn about [building Web apps with ASP.NET Core](#).

← → ↻ <https://localhost:7064/Employees/Create>

EmployeeCRUD Home Privacy

Create

Employee

Name

Salary

[Create](#)

[Back to List](#)

Index - EmployeeCRUD × +

← ↻ <https://localhost:7064/Employees>

EmployeeCRUD Home Privacy

Index

[Create New](#)

Name	Salary	
divs	50000	Edit Details Delete

D) Authentication and Authorization in ASP.NET Core.

Algorithm:

Step1: Start the application.

Step2: Configure Cookie Authentication in **Program.cs**.

Step3: Enable Authentication and Authorization middleware.

Step4: Display the Login page.

Step5: Accept Username and Password from the user.

Step6: Validate the entered credentials.

Step7: If credentials are valid, create authentication claims.

Step8: Sign in the user using Cookie Authentication.

Step9: Redirect the user to the secured page.

Step10: If credentials are invalid, display an error message.

Step11: Allow the authenticated user to log out.

Code:

program.cs

```
using Microsoft.AspNetCore.Authentication.Cookies;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Home/Index";
    });
builder.Services.AddAuthorization();
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

homecontroller.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;
namespace AuthDemo.Controllers
{
    0 references
    public class HomeController : Controller
    {
        0 references
        public IActionResult Index()
        {
            return View();
        }
        [HttpPost]

        public async Task<IActionResult> Index(string username, string password)
        {
            if (username == "Divyanjali" && password == "123")
            {
                var claims = new List<Claim>
                {
                    new Claim(ClaimTypes.Name, username)
                };
                var identity = new ClaimsIdentity(claims,
                    CookieAuthenticationDefaults.AuthenticationScheme);
                await HttpContext.SignInAsync(
                    CookieAuthenticationDefaults.AuthenticationScheme,
                    new ClaimsPrincipal(identity));
                return RedirectToAction("Secure");
            }
            ViewBag.Message = "Invalid Username or Password";
            return View();
        }
        [Authorize]
```

```
public IActionResult Secure()
{
    return Content("Welcome! Login Successful.");
}

0 references
public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(
        CookieAuthenticationDefaults.AuthenticationScheme);
    return RedirectToAction("Index");
}
}
```

index.cshtml

```
<h2>Login</h2>
<form method="post">
    Username :
    <input type="text" name="username" /><br />
    Password :
    <input type="password" name="password" /><br />
    <input type="submit" value="Login" />
</form>
<p style="color: red">
    @ViewBag.Message
</p>
```

Output:

Login

Username :

Password :

Login

Welcome! Login Successful.